



Entrada y salida de datos

Análisis, algoritmos y codificación para Dev-C++

Alumno: Emilio Abraham Castillo Urdiales
Matrícula: 2194561
Grupo: 011 A2026
Programa: Ingeniería en Administración y Sistemas
Materia: Aplicación práctica de la programación
Docente: Karla Patricia Uribe Sierra
Periodo: Agosto–diciembre de 2026

Fecha de realización: 16 de agosto de 2026
San Nicolás de los Garza, Nuevo León, México

Resumen

Se resuelven diecinueve problemas de entrada, proceso y salida en C++. Cada solución identifica datos, operación matemática, validaciones y resultado. Los archivos son independientes y compatibles con el compilador GNU C++ de Dev-C++. Los tipos **double** conservan la precisión decimal; las constantes identifican conversiones y magnitudes físicas; y los mensajes orientan la captura de datos.

Índice de Contenidos

1. Introducción	1
2. Programa 1. Área de un cuadrado	1
2.1. Instrucción	1
2.2. Análisis del problema	1
2.3. Algoritmo	1
2.4. Codificación del programa	1
2.5. Salida de consola	2
3. Programa 2. Área de un cuadrado con pow	2
3.1. Instrucción	2
3.2. Análisis del problema	2
3.3. Algoritmo	2
3.4. Codificación del programa	3
3.5. Salida de consola	3
4. Programa 3. Área de un círculo	4
4.1. Instrucción	4
4.2. Análisis del problema	4
4.3. Algoritmo	4
4.4. Codificación del programa	4
4.5. Salida de consola	5
5. Programa 4. Área de un triángulo	5
5.1. Instrucción	5
5.2. Análisis del problema	5
5.3. Algoritmo	5
5.4. Codificación del programa	5
5.5. Salida de consola	6
6. Programa 5. Hipotenusa	6
6.1. Instrucción	6
6.2. Análisis del problema	7
6.3. Algoritmo	7

6.4.	Codificación del programa	7
6.5.	Salida de consola	8
7.	Programa 6. Nombre y semestre	8
7.1.	Instrucción	8
7.2.	Análisis del problema	8
7.3.	Algoritmo	8
7.4.	Codificación del programa	8
7.5.	Salida de consola	9
8.	Programa 7. Fahrenheit a centígrados	9
8.1.	Instrucción	9
8.2.	Análisis del problema	10
8.3.	Algoritmo	10
8.4.	Codificación del programa	10
8.5.	Salida de consola	10
9.	Programa 8. Pies a metros	11
9.1.	Instrucción	11
9.2.	Análisis del problema	11
9.3.	Algoritmo	11
9.4.	Codificación del programa	11
9.5.	Salida de consola	12
10.	Programa 9. Libras a kilogramos	12
10.1.	Instrucción	12
10.2.	Análisis del problema	12
10.3.	Algoritmo	12
10.4.	Codificación del programa	13
10.5.	Salida de consola	13
11.	Programa 10. Acres a hectáreas	13
11.1.	Instrucción	14
11.2.	Análisis del problema	14
11.3.	Algoritmo	14
11.4.	Codificación del programa	14
11.5.	Salida de consola	15

12. Programa 11. Fuerza	15
12.1. Instrucción	15
12.2. Análisis del problema	15
12.3. Algoritmo	15
12.4. Codificación del programa	16
12.5. Salida de consola	16
13. Programa 12. Velocidad final de impacto	16
13.1. Instrucción	17
13.2. Análisis del problema	17
13.3. Algoritmo	17
13.4. Codificación del programa	17
13.5. Salida de consola	18
14. Programa 13. Energía potencial	18
14.1. Instrucción	18
14.2. Análisis del problema	18
14.3. Algoritmo	18
14.4. Codificación del programa	18
14.5. Salida de consola	19
15. Programa 14. Energía cinética	19
15.1. Instrucción	19
15.2. Análisis del problema	20
15.3. Algoritmo	20
15.4. Codificación del programa	20
15.5. Salida de consola	21
16. Programa 15. Hipotenusa dados los catetos	21
16.1. Instrucción	21
16.2. Análisis del problema	21
16.3. Algoritmo	21
16.4. Codificación del programa	21
16.5. Salida de consola	22
17. Programa 16. Volumen de una esfera	22
17.1. Instrucción	22

17.2. Análisis del problema	23
17.3. Algoritmo	23
17.4. Codificación del programa	23
17.5. Salida de consola	24
18. Programa 17. Recibo de la compañía de luz	24
18.1. Instrucción	24
18.2. Análisis del problema	24
18.3. Algoritmo	24
18.4. Codificación del programa	24
18.5. Salida de consola	25
19. Programa 18. Conversión conjunta de pies y libras	26
19.1. Instrucción	26
19.2. Análisis del problema	26
19.3. Algoritmo	26
19.4. Codificación del programa	26
19.5. Salida de consola	27
20. Programa 19. Área mediante la fórmula de Herón	27
20.1. Instrucción	27
20.2. Análisis del problema	28
20.3. Algoritmo	28
20.4. Codificación del programa	28
20.5. Salida de consola	29
21. Pseudocódigo completo	29
22. Resumen	35
23. Conclusiones	35

Índice de Códigos

1. Código C++ compatible con Dev-C++.	1
2. Salida de consola.	2
3. Código C++ compatible con Dev-C++.	3

4.	Salida de consola.	3
5.	Código C++ compatible con Dev-C++.	4
6.	Salida de consola.	5
7.	Código C++ compatible con Dev-C++.	6
8.	Salida de consola.	6
9.	Código C++ compatible con Dev-C++.	7
10.	Salida de consola.	8
11.	Código C++ compatible con Dev-C++.	8
12.	Salida de consola.	9
13.	Código C++ compatible con Dev-C++.	10
14.	Salida de consola.	10
15.	Código C++ compatible con Dev-C++.	11
16.	Salida de consola.	12
17.	Código C++ compatible con Dev-C++.	13
18.	Salida de consola.	13
19.	Código C++ compatible con Dev-C++.	14
20.	Salida de consola.	15
21.	Código C++ compatible con Dev-C++.	16
22.	Salida de consola.	16
23.	Código C++ compatible con Dev-C++.	17
24.	Salida de consola.	18
25.	Código C++ compatible con Dev-C++.	19
26.	Salida de consola.	19
27.	Código C++ compatible con Dev-C++.	20
28.	Salida de consola.	21
29.	Código C++ compatible con Dev-C++.	21
30.	Salida de consola.	22
31.	Código C++ compatible con Dev-C++.	23
32.	Salida de consola.	24
33.	Código C++ compatible con Dev-C++.	25
34.	Salida de consola.	25
35.	Código C++ compatible con Dev-C++.	26
36.	Salida de consola.	27
37.	Código C++ compatible con Dev-C++.	28
38.	Salida de consola.	29

39.	Pseudocódigo PSeInt de los 19 problemas.	29
-----	--	----

1. Introducción

El modelo entrada–proceso–salida organiza cada solución. La entrada reúne los valores del usuario, el proceso aplica fórmulas o conversiones y la salida comunica el resultado. El análisis previo determina datos, unidades, ecuaciones y condiciones de validez para problemas geométricos, físicos, de conversión y de administración.

2. Programa 1. Área de un cuadrado

2.1. Instrucción

Calcular e imprimir el área de un cuadrado a partir de la longitud de su lado.

2.2. Análisis del problema

La entrada es el lado l , que debe ser positivo. El proceso aplica $A = l \times l$ y la salida es el área en unidades cuadradas.

2.3. Algoritmo

1. Solicitar el lado.
2. Verificar que sea mayor que cero.
3. Calcular $A = l \times l$.
4. Mostrar el área con dos decimales.

2.4. Codificación del programa

Código 1: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     double lado;
```

```
7  cout << "AREA DE UN CUADRADO\n";
8  cout << "Ingresa la medida del lado: ";
9  cin >> lado;
10 if (lado <= 0) {
11     cerr << "Error: el lado debe ser mayor que cero.\n";
12     return 1;
13 }
14 cout << fixed << setprecision(2)
15     << "Area = " << lado * lado << " unidades cuadradas\n";
16 return 0;
17 }
```

2.5. Salida de consola

Código 2: Salida de consola.

```
1 > Entrada: 5
2 AREA DE UN CUADRADO
3 Ingresa la medida del lado: Area = 25.00 unidades cuadradas
4 > Proceso finalizado con codigo 0
```

3. Programa 2. Área de un cuadrado con pow

3.1. Instrucción

Calcular el área de un cuadrado mediante la función de potencia.

3.2. Análisis del problema

Se recibe el lado l y se eleva al cuadrado con `pow(lado, 2)`. El uso de `<cmath>` permite practicar funciones matemáticas de la biblioteca estándar.

3.3. Algoritmo

1. Leer el lado.
2. Validar que sea positivo.

3. Calcular $A = l^2$ con `pow`.

4. Imprimir el resultado.

3.4. Codificación del programa

Código 3: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     double lado;
8     cout << "AREA DE UN CUADRADO CON POW\n";
9     cout << "Ingresa la medida del lado: ";
10    cin >> lado;
11    if (lado <= 0) {
12        cerr << "Error: el lado debe ser mayor que cero.\n";
13        return 1;
14    }
15    double area = pow(lado, 2.0);
16    cout << fixed << setprecision(2)
17         << "Area = " << area << " unidades cuadradas\n";
18    return 0;
19 }
```

3.5. Salida de consola

Código 4: Salida de consola.

```
1 > Entrada: 5
2 AREA DE UN CUADRADO CON POW
3 Ingresa la medida del lado: Area = 25.00 unidades cuadradas
4 > Proceso finalizado con codigo 0
```

4. Programa 3. Área de un círculo

4.1. Instrucción

Calcular e imprimir el área de un círculo.

4.2. Análisis del problema

La entrada es el radio r . Se obtiene una aproximación portable de π con $\text{acos}(-1)$ y se aplica $A = \pi r^2$. Un radio no positivo se considera inválido.

4.3. Algoritmo

1. Solicitar el radio.
2. Validar $r > 0$.
3. Calcular $A = \pi r^2$.
4. Mostrar el área.

4.4. Codificación del programa

Código 5: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     const double PI = acos(-1.0);
8     double radio;
9     cout << "AREA DE UN CIRCULO\n";
10    cout << "Ingresa el radio: ";
11    cin >> radio;
12    if (radio <= 0) {
13        cerr << "Error: el radio debe ser mayor que cero.\n";
14        return 1;
15    }
```

```
15 }  
16 double area = PI * pow(radio, 2.0);  
17 cout << fixed << setprecision(2)  
18     << "Area = " << area << " unidades cuadradas\n";  
19 return 0;  
20 }
```

4.5. Salida de consola

Código 6: Salida de consola.

```
1 > Entrada: 2  
2 AREA DE UN CIRCULO  
3 Ingresa el radio: Area = 12.57 unidades cuadradas  
4 > Proceso finalizado con codigo 0
```

5. Programa 4. Área de un triángulo

5.1. Instrucción

Calcular el área de un triángulo a partir de su base y altura.

5.2. Análisis del problema

Las entradas son base b y altura h , ambas positivas. El proceso es $A = (b \times h)/2$.

5.3. Algoritmo

1. Leer base y altura.
2. Comprobar que sean positivas.
3. Calcular $A = bh/2$.
4. Imprimir el área.

5.4. Codificación del programa

Código 7: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     double base, altura;
7     cout << "AREA DE UN TRIANGULO\n";
8     cout << "Ingresa la base: ";
9     cin >> base;
10    cout << "Ingresa la altura: ";
11    cin >> altura;
12    if (base <= 0 || altura <= 0) {
13        cerr << "Error: base y altura deben ser mayores que cero.\n";
14        return 1;
15    }
16    double area = base * altura / 2.0;
17    cout << fixed << setprecision(2)
18        << "Area = " << area << " unidades cuadradas\n";
19    return 0;
20 }
```

5.5. Salida de consola

Código 8: Salida de consola.

```
1 > Entrada: 10, 6
2 AREA DE UN TRIANGULO
3 Ingresa la base: Ingresa la altura: Area = 30.00 unidades cuadradas
4 > Proceso finalizado con codigo 0
```

6. Programa 5. Hipotenusa

6.1. Instrucción

Calcular la hipotenusa de un triángulo rectángulo.

6.2. Análisis del problema

Los datos son los catetos a y b . Por el teorema de Pitágoras, la hipotenusa es $c = \sqrt{a^2 + b^2}$.

6.3. Algoritmo

1. Leer ambos catetos.
2. Validar que sean positivos.
3. Calcular $c = \sqrt{a^2 + b^2}$.
4. Mostrar la hipotenusa.

6.4. Codificación del programa

Código 9: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     double catetoA, catetoB;
8     cout << "HIPOTENUSA DE UN TRIANGULO RECTANGULO\n";
9     cout << "Ingresa el primer cateto: ";
10    cin >> catetoA;
11    cout << "Ingresa el segundo cateto: ";
12    cin >> catetoB;
13    if (catetoA <= 0 || catetoB <= 0) {
14        cerr << "Error: los catetos deben ser mayores que cero.\n";
15        return 1;
16    }
17    double hipotenusa = sqrt(pow(catetoA, 2.0) + pow(catetoB, 2.0));
18    cout << fixed << setprecision(2)
19         << "Hipotenusa = " << hipotenusa << '\n';
20    return 0;
21 }
```

6.5. Salida de consola

Código 10: Salida de consola.

```
1 > Entrada: 3, 4
2 HIPOTENUSA DE UN TRIANGULO RECTANGULO
3 Ingresa el primer cateto: Ingresa el segundo cateto: Hipotenusa = 5.00
4 > Proceso finalizado con codigo 0
```

7. Programa 6. Nombre y semestre

7.1. Instrucción

Ingresar el nombre del estudiante y el semestre que cursará, e imprimir ambos datos.

7.2. Análisis del problema

La entrada combina una cadena de texto y un entero. `getline` conserva los espacios del nombre completo. Se rechaza un nombre vacío o un semestre no positivo.

7.3. Algoritmo

1. Leer el nombre completo.
2. Leer el semestre.
3. Validar los datos.
4. Imprimir nombre y semestre.

7.4. Codificación del programa

Código 11: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
```



```
5 int main() {
6     string nombre;
7     int semestre;
8     cout << "DATOS DEL ESTUDIANTE\n";
9     cout << "Ingresa tu nombre completo: ";
10    getline(cin, nombre);
11    cout << "Ingresa el semestre que cursaras: ";
12    cin >> semestre;
13    if (nombre.empty() || semestre <= 0) {
14        cerr << "Error: proporciona un nombre y un semestre valido.\n";
15        return 1;
16    }
17    cout << "Nombre: " << nombre << '\n'
18        << "Semestre a cursar: " << semestre << '\n';
19    return 0;
20 }
```

7.5. Salida de consola

Código 12: Salida de consola.

```
1 > Entrada: Emilio Abraham Castillo Urdiales, 2
2 DATOS DEL ESTUDIANTE
3 Ingresa tu nombre completo: Ingresa el semestre que cursaras: Nombre: Emilio Abraham Castillo
   Urdiales
4 Semestre a cursar: 2
5 > Proceso finalizado con codigo 0
```

8. Programa 7. Fahrenheit a centígrados

8.1. Instrucción

Convertir una temperatura de grados Fahrenheit a grados centígrados.

8.2. Análisis del problema

La entrada es F y el proceso utiliza $C = (F - 32) \times 5/9$. Se emplean constantes decimales para impedir una división entera.

8.3. Algoritmo

1. Leer la temperatura Fahrenheit.
2. Calcular $C = (F - 32)5/9$.
3. Mostrar Fahrenheit y centígrados.

8.4. Codificación del programa

Código 13: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     double fahrenheit;
7     cout << "CONVERSION DE FAHRENHEIT A CENTIGRADOS\n";
8     cout << "Ingresa la temperatura en grados Fahrenheit: ";
9     cin >> fahrenheit;
10    double centigrados = (fahrenheit - 32.0) * 5.0 / 9.0;
11    cout << fixed << setprecision(2)
12         << fahrenheit << " F = " << centigrados << " C\n";
13    return 0;
14 }
```

8.5. Salida de consola

Código 14: Salida de consola.

```
1 > Entrada: 32
2 CONVERSION DE FAHRENHEIT A CENTIGRADOS
3 Ingresa la temperatura en grados Fahrenheit: 32.00 F = 0.00 C
```

```
4 > Proceso finalizado con código 0
```

9. Programa 8. Pies a metros

9.1. Instrucción

Convertir una longitud expresada en pies a metros.

9.2. Análisis del problema

Se usa la equivalencia exacta indicada: un pie equivale a 0.3048 metros. Por tanto, $m = p \times 0.3048$.

9.3. Algoritmo

1. Leer la longitud en pies.
2. Verificar que no sea negativa.
3. Multiplicar por 0.3048.
4. Imprimir los metros.

9.4. Codificación del programa

Código 15: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     const double METROS_POR_PIE = 0.3048;
7     double pies;
8     cout << "CONVERSION DE PIES A METROS\n";
9     cout << "Ingresa la longitud en pies: ";
10    cin >> pies;
11    if (pies < 0) {
```

```
12     cerr << "Error: la longitud no puede ser negativa.\n";
13     return 1;
14 }
15 cout << fixed << setprecision(4)
16     << pies << " pies = " << pies * METROS_POR_PIE << " metros\n";
17 return 0;
18 }
```

9.5. Salida de consola

Código 16: Salida de consola.

```
1 > Entrada: 10
2 CONVERSION DE PIES A METROS
3 Ingresa la longitud en pies: 10.0000 pies = 3.0480 metros
4 > Proceso finalizado con codigo 0
```

10. Programa 9. Libras a kilogramos

10.1. Instrucción

Convertir un peso expresado en libras a kilogramos.

10.2. Análisis del problema

La entrada son libras y se aplica $kg = lb \times 0.45359237$. La salida conserva cuatro decimales para mostrar adecuadamente la conversión.

10.3. Algoritmo

1. Leer las libras.
2. Validar que no sean negativas.
3. Multiplicar por 0.45359237.
4. Mostrar los kilogramos.

10.4. Codificación del programa

Código 17: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     const double KILOGRAMOS_POR_LIBRA = 0.45359237;
7     double libras;
8     cout << "CONVERSION DE LIBRAS A KILOGRAMOS\n";
9     cout << "Ingresa el peso en libras: ";
10    cin >> libras;
11    if (libras < 0) {
12        cerr << "Error: el peso no puede ser negativo.\n";
13        return 1;
14    }
15    cout << fixed << setprecision(4)
16         << libras << " lb = " << libras * KILOGRAMOS_POR_LIBRA << " kg\n";
17    return 0;
18 }
```

10.5. Salida de consola

Código 18: Salida de consola.

```
1 > Entrada: 10
2 CONVERSION DE LIBRAS A KILOGRAMOS
3 Ingresa el peso en libras: 10.0000 lb = 4.5359 kg
4 > Proceso finalizado con codigo 0
```

11. Programa 10. Acres a hectáreas

11.1. Instrucción

Introducir la extensión de un terreno en acres y calcularla en hectáreas, considerando 1 acre = 4047 m^2 y 1 hectárea = 10000 m^2 .

11.2. Análisis del problema

Primero se convierten acres a metros cuadrados y después se dividen entre los metros cuadrados de una hectárea: $ha = acres \times 4047/10000$.

11.3. Algoritmo

1. Leer acres.
2. Validar que no sean negativos.
3. Convertir a metros cuadrados.
4. Dividir entre 10000.
5. Imprimir hectáreas.

11.4. Codificación del programa

Código 19: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     const double METROS_CUADRADOS_POR_ACRE = 4047.0;
7     const double METROS_CUADRADOS_POR_HECTAREA = 10000.0;
8     double acres;
9     cout << "CONVERSION DE ACRES A HECTAREAS\n";
10    cout << "Ingresa la extension en acres: ";
11    cin >> acres;
12    if (acres < 0) {
13        cerr << "Error: la extension no puede ser negativa.\n";
14        return 1;
15    }
```

```
15 }  
16 double hectareas = acres * METROS_CUADRADOS_POR_ACRE  
17     / METROS_CUADRADOS_POR_HECTAREA;  
18 cout << fixed << setprecision(4)  
19     << acres << " acres = " << hectareas << " hectareas\n";  
20 return 0;  
21 }
```

11.5. Salida de consola

Código 20: Salida de consola.

```
1 > Entrada: 1  
2 CONVERSION DE ACRES A HECTAREAS  
3 Ingresa la extension en acres: 1.0000 acres = 0.4047 hectareas  
4 > Proceso finalizado con codigo 0
```

12. Programa 11. Fuerza

12.1. Instrucción

Calcular la fuerza aplicada a un cuerpo.

12.2. Análisis del problema

De acuerdo con la segunda ley de Newton, $F = m \times a$. Se ingresan masa en kilogramos y aceleración en m/s^2 ; el resultado se expresa en newtons.

12.3. Algoritmo

1. Leer masa y aceleración.
2. Validar que la masa no sea negativa.
3. Calcular $F = ma$.
4. Mostrar la fuerza en newtons.

12.4. Codificación del programa

Código 21: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     double masa, aceleracion;
7     cout << "CALCULO DE FUERZA\n";
8     cout << "Ingresa la masa en kilogramos: ";
9     cin >> masa;
10    cout << "Ingresa la aceleracion en m/s^2: ";
11    cin >> aceleracion;
12    if (masa < 0) {
13        cerr << "Error: la masa no puede ser negativa.\n";
14        return 1;
15    }
16    double fuerza = masa * aceleracion;
17    cout << fixed << setprecision(2)
18         << "Fuerza = " << fuerza << " newtons\n";
19    return 0;
20 }
```

12.5. Salida de consola

Código 22: Salida de consola.

```
1 > Entrada: 10, 2
2 CALCULO DE FUERZA
3 Ingresa la masa en kilogramos: Ingresa la aceleracion en m/s^2: Fuerza = 20.00 newtons
4 > Proceso finalizado con codigo 0
```

13. Programa 12. Velocidad final de impacto

13.1. Instrucción

Calcular la velocidad final de impacto de un cuerpo en caída libre desde el reposo.

13.2. Análisis del problema

Sin resistencia del aire y con velocidad inicial cero, se aplica $v = \sqrt{2gh}$, con $g = 9.81 \text{ m/s}^2$ y altura h no negativa.

13.3. Algoritmo

1. Leer la altura.
2. Verificar $h \geq 0$.
3. Calcular $v = \sqrt{2gh}$.
4. Imprimir la velocidad en m/s .

13.4. Codificación del programa

Código 23: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     const double GRAVEDAD = 9.81;
8     double altura;
9     cout << "VELOCIDAD FINAL DE IMPACTO EN CAIDA LIBRE\n";
10    cout << "Ingresa la altura en metros: ";
11    cin >> altura;
12    if (altura < 0) {
13        cerr << "Error: la altura no puede ser negativa.\n";
14        return 1;
15    }
16    double velocidad = sqrt(2.0 * GRAVEDAD * altura);
17    cout << fixed << setprecision(2)
```

```
18     << "Velocidad final = " << velocidad << " m/s\n";
19     return 0;
20 }
```

13.5. Salida de consola

Código 24: Salida de consola.

```
1 > Entrada: 20
2 VELOCIDAD FINAL DE IMPACTO EN CAIDA LIBRE
3 Ingresa la altura en metros: Velocidad final = 19.81 m/s
4 > Proceso finalizado con codigo 0
```

14. Programa 13. Energía potencial

14.1. Instrucción

Calcular la energía potencial de un cuerpo en caída libre.

14.2. Análisis del problema

Las entradas son masa m y altura h . Con $g = 9.81 \text{ m/s}^2$, la energía potencial gravitatoria es $E_p = mgh$ y se expresa en joules.

14.3. Algoritmo

1. Leer masa y altura.
2. Validar que no sean negativas.
3. Calcular $E_p = mgh$.
4. Mostrar joules.

14.4. Codificación del programa

Código 25: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     const double GRAVEDAD = 9.81;
7     double masa, altura;
8     cout << "ENERGIA POTENCIAL GRAVITATORIA\n";
9     cout << "Ingresa la masa en kilogramos: ";
10    cin >> masa;
11    cout << "Ingresa la altura en metros: ";
12    cin >> altura;
13    if (masa < 0 || altura < 0) {
14        cerr << "Error: masa y altura no pueden ser negativas.\n";
15        return 1;
16    }
17    double energia = masa * GRAVEDAD * altura;
18    cout << fixed << setprecision(2)
19        << "Energia potencial = " << energia << " joules\n";
20    return 0;
21 }
```

14.5. Salida de consola

Código 26: Salida de consola.

```
1 > Entrada: 10, 5
2 ENERGIA POTENCIAL GRAVITATORIA
3 Ingresa la masa en kilogramos: Ingresa la altura en metros: Energia potencial = 490.50 joules
4 > Proceso finalizado con codigo 0
```

15. Programa 14. Energía cinética

15.1. Instrucción

Calcular la energía cinética de un cuerpo en movimiento.

15.2. Análisis del problema

Se ingresan masa m y velocidad v . El cálculo corresponde a $E_c = mv^2/2$. La velocidad puede tener signo, pero al elevarla al cuadrado la energía permanece no negativa.

15.3. Algoritmo

1. Leer masa y velocidad.
2. Validar la masa.
3. Calcular $E_c = mv^2/2$.
4. Mostrar joules.

15.4. Codificación del programa

Código 27: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     double masa, velocidad;
8     cout << "ENERGIA CINETICA\n";
9     cout << "Ingresa la masa en kilogramos: ";
10    cin >> masa;
11    cout << "Ingresa la velocidad en m/s: ";
12    cin >> velocidad;
13    if (masa < 0) {
14        cerr << "Error: la masa no puede ser negativa.\n";
15        return 1;
16    }
17    double energia = masa * pow(velocidad, 2.0) / 2.0;
18    cout << fixed << setprecision(2)
19         << "Energia cinetica = " << energia << " joules\n";
20    return 0;
21 }
```

15.5. Salida de consola

Código 28: Salida de consola.

```
1 > Entrada: 10, 3
2 ENERGIA CINETICA
3 Ingresa la masa en kilogramos: Ingresa la velocidad en m/s: Energia cinetica = 45.00 joules
4 > Proceso finalizado con codigo 0
```

16. Programa 15. Hipotenusa dados los catetos

16.1. Instrucción

Dados los catetos de un triángulo rectángulo, calcular e imprimir su hipotenusa.

16.2. Análisis del problema

Esta variante utiliza `hypot`, función estándar que calcula directamente $\sqrt{a^2 + b^2}$ y mejora la claridad del código.

16.3. Algoritmo

1. Leer los catetos.
2. Comprobar que sean positivos.
3. Evaluar `hypot(a,b)`.
4. Imprimir el resultado.

16.4. Codificación del programa

Código 29: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
```

```
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     double cateto1, cateto2;
8     cout << "HIPOTENUSA A PARTIR DE LOS CATETOS\n";
9     cout << "Ingresa el cateto 1: ";
10    cin >> cateto1;
11    cout << "Ingresa el cateto 2: ";
12    cin >> cateto2;
13    if (cateto1 <= 0 || cateto2 <= 0) {
14        cerr << "Error: los catetos deben ser mayores que cero.\n";
15        return 1;
16    }
17    double hipotenusa = hypot(cateto1, cateto2);
18    cout << fixed << setprecision(2)
19         << "Hipotenusa = " << hipotenusa << '\n';
20    return 0;
21 }
```

16.5. Salida de consola

Código 30: Salida de consola.

```
1 > Entrada: 3, 4
2 HIPOTENUSA A PARTIR DE LOS CATETOS
3 Ingresa el cateto 1: Ingresa el cateto 2: Hipotenusa = 5.00
4 > Proceso finalizado con codigo 0
```

17. Programa 16. Volumen de una esfera

17.1. Instrucción

Calcular e imprimir el volumen de una esfera.

17.2. Análisis del problema

La entrada es el radio $r > 0$. El volumen se obtiene con $V = (4/3)\pi r^3$. Se escribe 4.0/3.0 para conservar la parte decimal.

17.3. Algoritmo

1. Leer el radio.
2. Validar que sea positivo.
3. Calcular $V = (4/3)\pi r^3$.
4. Mostrar unidades cúbicas.

17.4. Codificación del programa

Código 31: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     const double PI = acos(-1.0);
8     double radio;
9     cout << "VOLUMEN DE UNA ESFERA\n";
10    cout << "Ingresa el radio: ";
11    cin >> radio;
12    if (radio <= 0) {
13        cerr << "Error: el radio debe ser mayor que cero.\n";
14        return 1;
15    }
16    double volumen = (4.0 / 3.0) * PI * pow(radio, 3.0);
17    cout << fixed << setprecision(2)
18         << "Volumen = " << volumen << " unidades cubicas\n";
19    return 0;
20 }
```

17.5. Salida de consola

Código 32: Salida de consola.

```
1 > Entrada: 3
2 VOLUMEN DE UNA ESFERA
3 Ingresa el radio: Volumen = 113.10 unidades cubicas
4 > Proceso finalizado con codigo 0
```

18. Programa 17. Recibo de la compañía de luz

18.1. Instrucción

Generar el recibo de un cliente con su nombre, consumo en kilowatts y tarifa por kilowatt; el recibo debe mostrar el nombre y el pago.

18.2. Análisis del problema

El pago se calcula como $P = consumo \times tarifa$. Se usa `getline` para aceptar nombres completos y se impiden consumo o tarifa negativos.

18.3. Algoritmo

1. Leer el nombre del cliente.
2. Leer consumo y tarifa.
3. Validar las entradas.
4. Calcular el pago.
5. Imprimir nombre y total con dos decimales.

18.4. Codificación del programa

Código 33: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <string>
4 using namespace std;
5
6 int main() {
7     string nombre;
8     double consumo, tarifa;
9     cout << "RECIBO DE ENERGIA ELECTRICA\n";
10    cout << "Ingresa el nombre del cliente: ";
11    getline(cin, nombre);
12    cout << "Ingresa el consumo en kilowatts: ";
13    cin >> consumo;
14    cout << "Ingresa la tarifa por kilowatt: $";
15    cin >> tarifa;
16    if (nombre.empty() || consumo < 0 || tarifa < 0) {
17        cerr << "Error: revisa los datos proporcionados.\n";
18        return 1;
19    }
20    double pago = consumo * tarifa;
21    cout << "\n--- RECIBO ---\n"
22         << "Cliente: " << nombre << '\n'
23         << fixed << setprecision(2)
24         << "Pago: $" << pago << '\n';
25    return 0;
26 }
```

18.5. Salida de consola

Código 34: Salida de consola.

```
1 > Entrada: Emilio Castillo, 250, 0.85
2 RECIBO DE ENERGIA ELECTRICA
3 Ingresa el nombre del cliente: Ingresa el consumo en kilowatts: Ingresa la tarifa por kilowatt: $
4 --- RECIBO ---
5 Cliente: Emilio Castillo
6 Pago: $212.50
```

```
7 > Proceso finalizado con código 0
```

19. Programa 18. Conversión conjunta de pies y libras

19.1. Instrucción

Introducir longitud en pies y peso en libras, e imprimir metros y kilogramos.

19.2. Análisis del problema

Se resuelven dos conversiones independientes: $m = \text{pies} \times 0.3048$ y $kg = \text{libras} \times 0.45359237$.

19.3. Algoritmo

1. Leer pies y libras.
2. Verificar que no sean negativos.
3. Convertir pies a metros.
4. Convertir libras a kilogramos.
5. Mostrar ambos resultados.

19.4. Codificación del programa

Código 35: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main() {
6     const double METROS_POR_PIE = 0.3048;
7     const double KILOGRAMOS_POR_LIBRA = 0.45359237;
```

```
8  double pies, libras;
9  cout << "CONVERSION DE LONGITUD Y PESO\n";
10 cout << "Ingresa la longitud en pies: ";
11 cin >> pies;
12 cout << "Ingresa el peso en libras: ";
13 cin >> libras;
14 if (pies < 0 || libras < 0) {
15     cerr << "Error: los valores no pueden ser negativos.\n";
16     return 1;
17 }
18 cout << fixed << setprecision(4)
19     << "Longitud: " << pies * METROS_POR_PIE << " metros\n"
20     << "Peso: " << libras * KILOGRAMOS_POR_LIBRA << " kilogramos\n";
21 return 0;
22 }
```

19.5. Salida de consola

Código 36: Salida de consola.

```
1 > Entrada: 10, 10
2 CONVERSION DE LONGITUD Y PESO
3 Ingresa la longitud en pies: Ingresa el peso en libras: Longitud: 3.0480 metros
4 Peso: 4.5359 kilogramos
5 > Proceso finalizado con codigo 0
```

20. Programa 19. Área mediante la fórmula de Herón

20.1. Instrucción

Leer los tres lados de un triángulo y calcular su área con la fórmula de Herón.

20.2. Análisis del problema

Antes del cálculo se valida la desigualdad triangular. Si los lados son válidos, el semiperímetro es $s = (a + b + c)/2$ y el área $A = \sqrt{s(s - a)(s - b)(s - c)}$.

20.3. Algoritmo

1. Leer a , b y c .
2. Verificar positividad y desigualdad triangular.
3. Calcular $s = (a + b + c)/2$.
4. Calcular la raíz de Herón.
5. Imprimir el área.

20.4. Codificación del programa

Código 37: Código C++ compatible con Dev-C++.

```
1 #include <iostream>
2 #include <iomanip>
3 #include <cmath>
4 using namespace std;
5
6 int main() {
7     double a, b, c;
8     cout << "AREA DE UN TRIANGULO: FORMULA DE HERON\n";
9     cout << "Ingresa el lado a: ";
10    cin >> a;
11    cout << "Ingresa el lado b: ";
12    cin >> b;
13    cout << "Ingresa el lado c: ";
14    cin >> c;
15    if (a <= 0 || b <= 0 || c <= 0 ||
16        a + b <= c || a + c <= b || b + c <= a) {
17        cerr << "Error: los valores no forman un triangulo valido.\n";
18        return 1;
19    }
```

```
20 double semiperimetro = (a + b + c) / 2.0;
21 double area = sqrt(semiperimetro * (semiperimetro - a)
22                 * (semiperimetro - b) * (semiperimetro - c));
23 cout << fixed << setprecision(2)
24      << "Area = " << area << " unidades cuadradas\n";
25 return 0;
26 }
```

20.5. Salida de consola

Código 38: Salida de consola.

```
1 > Entrada: 3, 4, 5
2 AREA DE UN TRIANGULO: FORMULA DE HERON
3 Ingresa el lado a: Ingresa el lado b: Ingresa el lado c: Area = 6.00 unidades cuadradas
4 > Proceso finalizado con codigo 0
```

21. Pseudocódigo completo

Los diecinueve algoritmos reproducen la entrada, validación, cálculo y salida de cada programa en sintaxis PSeInt.

Código 39: Pseudocódigo PSeInt de los 19 problemas.

```
1 // Pseudocodigos de la Actividad 2 - Entrada y salida de datos
2
3 Algoritmo AreaCuadrado
4   Escribir "Ingresa el lado"
5   Leer lado
6   Si lado <= 0 Entonces
7     Escribir "Error: lado invalido"
8   SiNo
9     area <- lado * lado
10    Escribir "Area = ", area
11  FinSi
12 FinAlgoritmo
13
14 Algoritmo AreaCuadradoPow
```

```
15  Escribir "Ingresa el lado"
16  Leer lado
17  Si lado <= 0 Entonces
18      Escribir "Error: lado invalido"
19  SiNo
20      area <- lado ^ 2
21      Escribir "Area = ", area
22  FinSi
23 FinAlgoritmo
24
25 Algoritmo AreaCirculo
26  PI <- 3.141592653589793
27  Escribir "Ingresa el radio"
28  Leer radio
29  Si radio <= 0 Entonces
30      Escribir "Error: radio invalido"
31  SiNo
32      area <- PI * radio ^ 2
33      Escribir "Area = ", area
34  FinSi
35 FinAlgoritmo
36
37 Algoritmo AreaTriangulo
38  Escribir "Ingresa base y altura"
39  Leer base, altura
40  Si base <= 0 O altura <= 0 Entonces
41      Escribir "Error: datos invalidos"
42  SiNo
43      area <- base * altura / 2
44      Escribir "Area = ", area
45  FinSi
46 FinAlgoritmo
47
48 Algoritmo Hipotenusa
49  Escribir "Ingresa los dos catetos"
50  Leer catetoA, catetoB
51  Si catetoA <= 0 O catetoB <= 0 Entonces
52      Escribir "Error: catetos invalidos"
53  SiNo
```

```
54   hipotenusa <- RC(catetoA ^ 2 + catetoB ^ 2)
55   Escribir "Hipotenusa = ", hipotenusa
56   FinSi
57 FinAlgoritmo
58
59 Algoritmo NombreSemestre
60   Escribir "Ingresa tu nombre completo"
61   Leer nombre
62   Escribir "Ingresa el semestre"
63   Leer semestre
64   Si semestre <= 0 Entonces
65     Escribir "Error: semestre invalido"
66   SiNo
67     Escribir "Nombre: ", nombre
68     Escribir "Semestre: ", semestre
69   FinSi
70 FinAlgoritmo
71
72 Algoritmo FahrenheitCentigrados
73   Escribir "Ingresa grados Fahrenheit"
74   Leer fahrenheit
75   centigrados <- (fahrenheit - 32) * 5 / 9
76   Escribir fahrenheit, " F = ", centigrados, " C"
77 FinAlgoritmo
78
79 Algoritmo PiesMetros
80   Escribir "Ingresa la longitud en pies"
81   Leer pies
82   Si pies < 0 Entonces
83     Escribir "Error: longitud invalida"
84   SiNo
85     metros <- pies * 0.3048
86     Escribir "Metros = ", metros
87   FinSi
88 FinAlgoritmo
89
90 Algoritmo LibrasKilogramos
91   Escribir "Ingresa el peso en libras"
92   Leer libras
```

```
93 Si libras < 0 Entonces
94   Escribir "Error: peso invalido"
95 SiNo
96   kilogramos <- libras * 0.45359237
97   Escribir "Kilogramos = ", kilogramos
98 FinSi
99 FinAlgoritmo
100
101 Algoritmo AcresHectareas
102   Escribir "Ingresa la extension en acres"
103   Leer acres
104   Si acres < 0 Entonces
105     Escribir "Error: extension invalida"
106   SiNo
107     hectareas <- acres * 4047 / 10000
108     Escribir "Hectareas = ", hectareas
109   FinSi
110 FinAlgoritmo
111
112 Algoritmo Fuerza
113   Escribir "Ingresa masa y aceleracion"
114   Leer masa, aceleracion
115   Si masa < 0 Entonces
116     Escribir "Error: masa invalida"
117   SiNo
118     fuerza <- masa * aceleracion
119     Escribir "Fuerza = ", fuerza, " N"
120   FinSi
121 FinAlgoritmo
122
123 Algoritmo VelocidadImpacto
124   gravedad <- 9.81
125   Escribir "Ingresa la altura"
126   Leer altura
127   Si altura < 0 Entonces
128     Escribir "Error: altura invalida"
129   SiNo
130     velocidad <- RC(2 * gravedad * altura)
131     Escribir "Velocidad = ", velocidad, " m/s"
```



```
132   FinSi
133 FinAlgoritmo
134
135 Algoritmo EnergiaPotencial
136   gravedad <- 9.81
137   Escribir "Ingresa masa y altura"
138   Leer masa, altura
139   Si masa < 0 O altura < 0 Entonces
140     Escribir "Error: datos invalidos"
141   SiNo
142     energia <- masa * gravedad * altura
143     Escribir "Energia potencial = ", energia, " J"
144   FinSi
145 FinAlgoritmo
146
147 Algoritmo EnergiaCinetica
148   Escribir "Ingresa masa y velocidad"
149   Leer masa, velocidad
150   Si masa < 0 Entonces
151     Escribir "Error: masa invalida"
152   SiNo
153     energia <- masa * velocidad ^ 2 / 2
154     Escribir "Energia cinetica = ", energia, " J"
155   FinSi
156 FinAlgoritmo
157
158 Algoritmo HipotenusaCatetos
159   Escribir "Ingresa los catetos"
160   Leer cateto1, cateto2
161   Si cateto1 <= 0 O cateto2 <= 0 Entonces
162     Escribir "Error: catetos invalidos"
163   SiNo
164     hipotenusa <- RC(cateto1 ^ 2 + cateto2 ^ 2)
165     Escribir "Hipotenusa = ", hipotenusa
166   FinSi
167 FinAlgoritmo
168
169 Algoritmo VolumenEsfera
170   PI <- 3.141592653589793
```

```
171  Escribir "Ingresa el radio"
172  Leer radio
173  Si radio <= 0 Entonces
174      Escribir "Error: radio invalido"
175  SiNo
176      volumen <- (4 / 3) * PI * radio ^ 3
177      Escribir "Volumen = ", volumen
178  FinSi
179 FinAlgoritmo
180
181 Algoritmo ReciboLuz
182     Escribir "Ingresa nombre, consumo y tarifa"
183     Leer nombre, consumo, tarifa
184     Si consumo < 0 O tarifa < 0 Entonces
185         Escribir "Error: datos invalidos"
186     SiNo
187         pago <- consumo * tarifa
188         Escribir "Cliente: ", nombre
189         Escribir "Pago: $", pago
190     FinSi
191 FinAlgoritmo
192
193 Algoritmo PiesLibras
194     Escribir "Ingresa pies y libras"
195     Leer pies, libras
196     Si pies < 0 O libras < 0 Entonces
197         Escribir "Error: datos invalidos"
198     SiNo
199         metros <- pies * 0.3048
200         kilogramos <- libras * 0.45359237
201         Escribir "Metros = ", metros
202         Escribir "Kilogramos = ", kilogramos
203     FinSi
204 FinAlgoritmo
205
206 Algoritmo AreaHeron
207     Escribir "Ingresa los lados a, b y c"
208     Leer a, b, c
209     Si a <= 0 O b <= 0 O c <= 0 O a+b <= c O a+c <= b O b+c <= a Entonces
```

```
210   Escribir "Error: triangulo invalido"
211   SiNo
212     semiperimetro <- (a + b + c) / 2
213     area <- RC(semiperimetro * (semiperimetro-a) * (semiperimetro-b) * (semiperimetro-c))
214     Escribir "Area = ", area
215   FinSi
216 FinAlgoritmo
```

22. Resumen

Entrada, proceso y salida. La programación expresa procedimientos mediante instrucciones precisas y ordenadas. Cada solución inicia con el análisis de las entradas, continúa con una fórmula o transformación y termina con una salida comprensible. El tipo **double** conserva valores decimales y las constantes con nombre identifican factores como la gravedad y las equivalencias de unidades.

Bibliotecas y validación. **iostream** administra la entrada y salida; **iomanip** controla los decimales; **cmath** proporciona potencias y raíces; y **string** almacena nombres completos. Las validaciones impiden cálculos con lados negativos, radios nulos o longitudes que no forman un triángulo.

Aplicación. Formular el algoritmo antes del código reduce errores y permite verificar la unidad y el propósito de cada variable. Los programas se compilan por separado en Dev-C++ y sus salidas de consola confirman los resultados de los casos evaluados.

23. Conclusiones

Los diecinueve programas muestran que el modelo entrada-proceso-salida permite resolver problemas diversos con una estructura común. El análisis previo ayudó a seleccionar fórmulas, tipos de datos y validaciones. El uso de funciones de la biblioteca estándar hizo los cálculos más claros y portables. Como mejora posterior pueden añadirse ciclos para repetir cada operación y controles para detectar entradas que no sean numéricas; sin embargo, las versiones entregadas satisfacen el objetivo de practicar instrucciones básicas y pueden compilarse individualmente en Dev-C++.